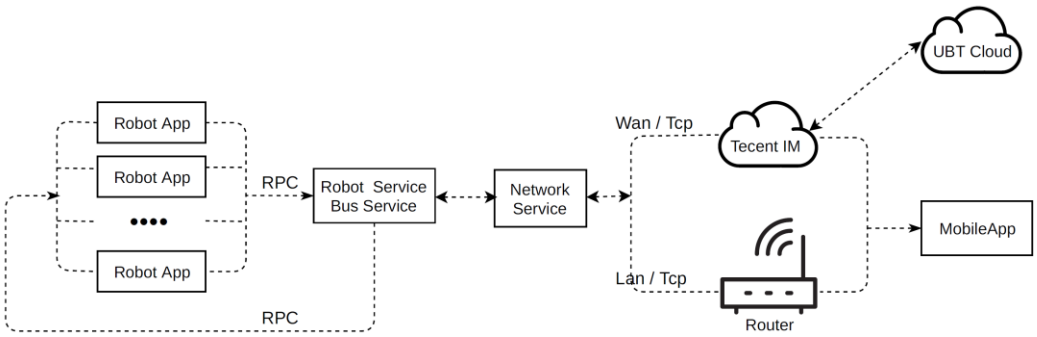
Description

- Setup secured and reliable connection between mobile App and robot.
- Enable mobile App to call applications and services from robot.
- Enable robot to push message to mobile App.

Service Structure



System Structure (*In case of Tencent IM and UBT Cloud server*)

Function Description:

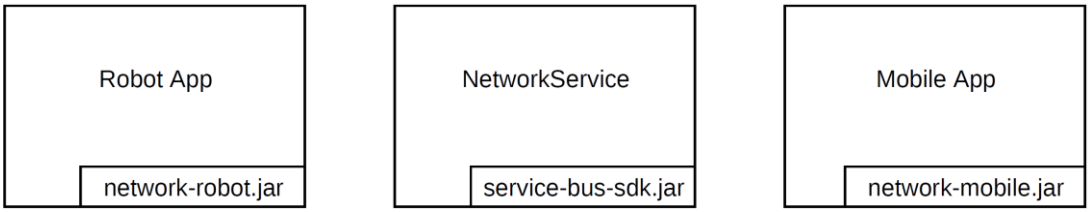
- Robot Service Master Bus Service
  - This service receives message from network service or between robot progress and forward subscribed message to related progress.
  - Robot progress can send message to this service, the service will forward the message to network service or to related progress.
  - To conclude, PacketHubService works as message Hub.

- Robot App
  - Each robot App implements individual functions: Dancing, music and so on. Binding Robot Service Bus Service and send/receive message to/from the service, the current progress can communicate with network service and other robot progress indirectly.
  - Binding, unbinding and rebinding Robot Service Bus Service and send/receive message to/from Robot Service Bus Service should be public for all robot progress using Robot Service Bus Service.
  - **A SDK is provided for robot App to integrated and send data to mobile App.**

- **Network Service**
  - This service receives message from external network (Cloud Server) or internal network (between mobile App and robot) and forward to Robot Service Bus Service; and receive message from Robot Service Bus Service and forward to mobile App or IM service (*could be different between IM service providers*).
  - Technical implementation should contains those 3 routes above.
  - Integrate Cloud communication SDK to setup connection to Cloud server.
  - Implement TCP Socket Server to setup connection to mobile App.
  - Implement Android Service and bind Robot Service Bus Service to communicate with robot progress

- Mobile App
  - Integrate Cloud communication SDK to setup connection to Cloud server.
  - Implement TCP Socket Server Client to setup connection to mobile App.
  - **A SDK is provided for Mobile App to integrated and send data to robot App.**

Module Dependencies:

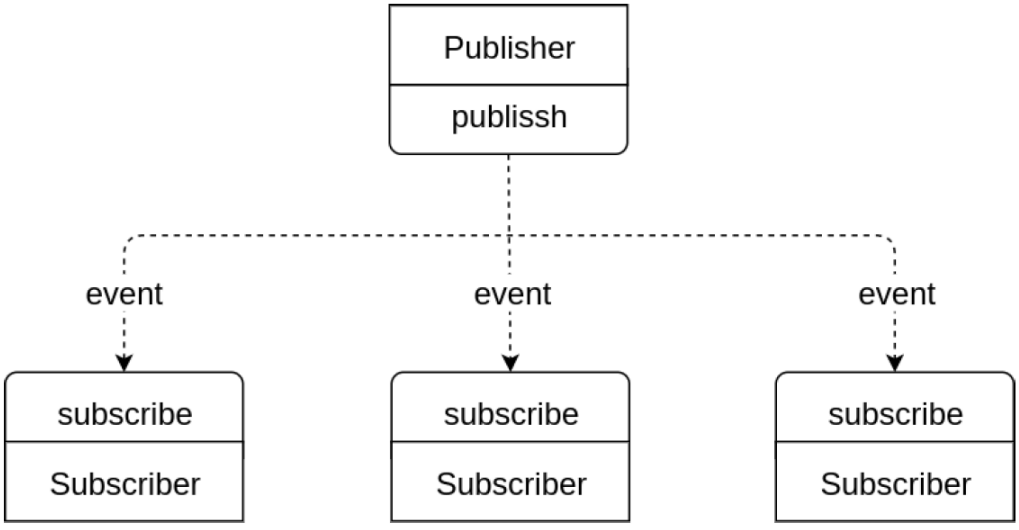


Library Dependencies diagram

Message Model

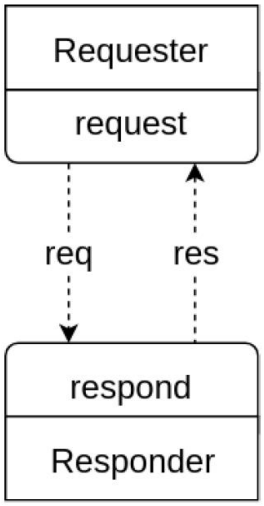
Same to Robot Service Bus Service

" Publish - Subscriber " Model



- Scenario
  - Publish event notification:1 -> N
  - No need acknowledgement

" Request - Respond " Model



- Scenario
  - 1 <-> 1，publish request and wait for respond
  - Need acknowledgement

Protocol

- One robot only support one connection to cellphone App via internal network at once
- Cellphone App do not support request registration
- Cellphone App do not support to send "publish" event

Communication-mobile SDK API Description

Get Network Channel

```
ChannelHub get(int type);
```

Support 3 network types

```
public interface NetType {
    int AUTO_HUB = 0; //auto select
    int LAN_HUB = 1; //LAN
    int WAN_HUB = 2; //IM
}
```

ChannelHub port

Initiate

```
void init(Context context);


void init(Context context, Config config);//LAN or IM configuration
```

connect

```
void connect(ConnectConfig connectConfig, ConnectionCallback callback);
```

Disconnect

```
void disconnect();
```

Register Connection State Listener

```
void registerConnectionStateListener(ConnectionStatusListener listener);
```

Get Connection State

```
State getConnectionState();
```

Robot Master retrieve

```
//Both Robot Master service and Master App is available for cellphone App

RobotService applications =

getBusService(RobotService.ALL_ROBOT_BUSINESS_APPLICATIONS) ;


applications.call(request, ResponseCallback callback);


RobotService with interface as listed：


Response call(String path) throws CallException; // sync call


Response call(String path, @Nullable Param param) throws CallException;


Cancelable call(String path, ResponseCallback callback); // unsync
```

```
call,cancelable



Cancelable call(String path, @Nullable Param param, @Nullable

ResponseCallback callback);
```

Retrieve Service

```
//same to main service SDK

RobotService getBusService(String serviceName) ;




service.call(request, ResponseCallback callback);
```

Exit

```
void release();//release resource and deregister listener and responder
```

Network-robot SDK API Description

Mobile App does not support request registration, only Robot App receives request from mobile App and respond to the request.

With Network-robot SDK, robot App can push message to mobile App.

```
void push(Msg msg);
```

“Push” Implementation

LAN

Network Service register a request,（path：/net/lan/push），robot App will send this request in Network Service before pushing message

IM

Based on IM send process.

Notice：Push only support robot App send message to mobile App, cannot send from mobile App to robot App

Internal network API（Network module based on this library)

```
//Initiate

void init(Context context);

//connect

void connect(ConnectConfig connectConfig);



void send(byte[] data);



void setDataReceiveListener(DataReceiveListener listener);
```

```
void registerConnectionStateListener(ConnectionStatusListener listener);

void registerClosedListener(ClosedListener listener);

State getConnectionState();

void disconnect();

void release();
```

Message Format

Data package is in TLV format，could be protobuf or gson types，currently, UBTECH official setting only support protobuf.

Data package： type(1Byte) + length(4Bytes) + data(nBytes)

Definition of protobuf is listed below：

packet.proto

```
syntax = "proto3";



package packet;



option java_package = "com.ubtrobot.lib.packethub.packet";

option java_outer_classname = "PacketProto";



import "google/protobuf/any.proto";



imMessage Packet {

    string id = 1;//id

    int64 when = 2;//time tag unit:ms

    string from = 3;//package name

    string version = 5;//version

    string body_type = 6;//body type

    google.protobuf.Any body = 7;
```

```
}
```

Body_type only support these types:

```
    //publish event

    String BODY_TYPE_PUB = "sub-pub.pub";



    //subscribe event

    String BODY_TYPE_SUB = "sub-pub.sub";

    //unsubscribe event

    String BODY_TYPE_UNSUB = "sub-pub.unsub";



    //request

    String BODY_TYPE_REQ = "req-res.req";

    //response

    String BODY_TYPE_RES = "req-res.res";



    //call robot service
```

```
    String BODY_TYPE_SERVICE_REQ = "call-service.req";



    //call response from robot service

    String BODY_TYPE_SERVICE_RES = "call-service.res";
```

“body” has two types:

```
// event

imMessage Event {

    // event id,

    string id = 1;

    // sender name, regulation:  “(mobile|robot|cloud:package_name)”

    string endpoint = 2;

    // event action

    string action = 3;

    // occurred time

    int64 when = 4;

    // parameters in the event
```

```
    google.protobuf.Any param = 5;

}



// request

imMessage Request {

    // request id

    string id = 1;

    // sender name, regulation:  "(mobile|robot|cloud:package_name)"

    string endpoint = 2;

    // request resource address

    string uri = 3;

    // occurred time

    int64 when = 4;

    // parameters in the request

    google.protobuf.Any param = 5;

}
```

Network Security

Authentication

Mobile app embedded with a public key，app will calculate a fixed data via Hash algorithm then encrypt by the public key. Mobile app send both the original data and encrypted data to the robot.

When the robot received the encrypted data and use private key to decrypt, also calculate the original data via Hash algorithm. If the decrypted data and Hashed data is equal, robot and mobile are authenticated each other as legal user.

Connection

Mobile App request symmetric encryption cipher and encrypt the public key before transaction.　Once Robot receive the request, robot uses private key to decrypted the request and randomly generate a symmetric encryption cipher. The generated cipher will be sent to mobile App after encrypted by private key.

User Authentication

LAN

- Direct connection
  - Robot as Wi-Fi hotpot，request Wi-Fi password to connect;

- Mobile App input Wi-Fi password to connect to the robot as a legal user.

- LAN
- Mobile App use public key to encrypted userId and send to the robot; Robot use private key to decrypted and check whether the user is binding to this robot.
- Otherwise，user input password to access to the robot（same to “direct connection”），the password is encrypted by public key and send to the robot, Robot use private key to decrypted and check whether the user is legal

IM

- Based on IM user authentication system